

RAG Pipeline – Observations & Analysis

Startup Funding RAG Application (Funding Navigator AI)

v18hub Cohort • LLM Behaviour • Chunking • Indexing • Retrieval Quality • Evaluations • Improvements

1. LLM Models Used

Model	Status	Notes
LLaMA 3 8B Instruct (Ollama)	Active	Used for answer generation with temperature tuning

LLM Behaviour Observations

What worked:

- Generated structured and context-aware answers using retrieved data
 - Handled retrieval-based inputs effectively
 - Lower temperature (0.3) produced more factual and deterministic responses
 - Medium temperature (0.7) provided a good balance between accuracy and explanation (used in final system)
-

Issues found:

- Tended to over-explain and include unnecessary details
 - Sometimes misinterpreted the exact intent of the query
 - Struggled to aggregate information across multiple retrieved chunks
-

Model Details:

- Model: meta-llama-3-8b-instruct.Q4_K_M.gguf
- Temperature: 0.7

- Context Window: ~8192 tokens
-

Characteristics

- Transformer-based autoregressive model
 - Instruction-tuned for question-answering tasks
 - Capable of reasoning and summarization
 - Generates responses based on retrieved context (RAG setup)
 - Runs locally using Ollama
-

Limitations

- Performance is dependent on the quality of retrieved context
 - May produce incomplete or partially accurate answers
 - Struggles with multi-chunk information synthesis
 - Computationally heavy due to large model size (8B parameters)
-

2. Chunking Strategy Observations

Recursive Character-Based Chunking

What worked:

- Used RecursiveCharacterTextSplitter with:
 - chunk_size = 1000
 - chunk_overlap = 200
 - Prioritized paragraph-level splitting (\n\n) for better semantic grouping
 - Maintained context continuity using overlap
 - Adaptable to varying document structures
-

Issues found:

- Structured content (e.g., lists, policy details) may be split across multiple chunks
 - Logical sections may break when exceeding chunk size
 - May require better handling for structured documents
-

3. Indexing Observations

HNSW – chunk_text_vector

- Index Type: **HNSW (Hierarchical Navigable Small World)**
- Metric Type: **COSINE similarity**

Parameters:

- M = 16
 - efConstruction = 200
-

Why this index was used

- Efficient for high-dimensional vector search
 - Provides fast and accurate nearest-neighbour retrieval
 - Suitable for semantic similarity tasks in RAG systems
 - Requires minimal tuning compared to other ANN methods
-

Metadata Indexing

- Index Type: **INVERTED index**

Fields:

- language
- is_active
- crawl_date

- doc_last_modified
-

Purpose

- Enables filtering and structured queries
 - Improves retrieval efficiency when applying constraints
-

4. Retrieval Quality Observations

Initial Issue

- Some chunks were missing embeddings
 - Resulted in incomplete or irrelevant retrieval results
-

Fix Applied

- Re-ingested cleaned data into the database
 - Ensured all chunks had embeddings before retrieval
-

After Fix

- Retrieval quality improved significantly
 - Relevant chunks were consistently returned
-

Observations

- High contextual precision (relevant chunks retrieved)
 - Moderate recall for multi-part or complex queries
 - Retrieval effectiveness depends on:
 - completeness of embeddings
 - quality of chunking
-

5. Evaluations

Metrics Used

- Answer Relevancy
 - Contextual Precision
 - Contextual Recall
 - Faithfulness
 - Overall Evaluation Score
-

Evaluation Observations

Answer Relevancy (~0.5)

- Answers were partially correct but lacked precision
 - Included extra or irrelevant information
 - Incomplete responses for list-based queries
-

Contextual Precision (~0.95 – 1.0)

- Retrieved chunks were highly relevant to queries
 - Indicates strong indexing and retrieval performance
-

Contextual Recall (0.5 – 1.0)

- Lower recall when:
 - information was distributed across chunks
 - dataset lacked complete coverage
-

Faithfulness (~0.5)

- Responses were mostly grounded in retrieved context
- Some deviations due to misinterpretation or incomplete synthesis

Key Insight

The retrieval system performs well, but the LLM struggles with precise interpretation and complete synthesis of retrieved information.

6. Recommended Improvements

- Improve prompt design to enhance answer precision
 - Increase chunk size (~1500–2000 characters)
 - Improve handling of structured content (lists, policies)
 - Use domain-specific or fine-tuned embedding models
 - Introduce re-ranking before LLM generation
 - Implement hybrid search (vector + keyword)
 - Improve dataset completeness
-

7. End-to-End Pipeline Performance Summary

Component	What Worked	Issue Observed
Cleaning	Effective noise removal	No major issues
Chunking	Flexible and adaptive	May fragment structured content
Embeddings (MiniLM)	Fast and efficient	Limited domain specificity
Indexing (HNSW)	Fast and accurate	No major issues
Retrieval	High precision	Moderate recall
LLM (LLaMA 3)	Context-aware responses	Moderate relevancy & faithfulness

8. Conclusion

Strengths

- Clean preprocessing improved embedding quality
 - Strong retrieval performance with high contextual precision
 - Efficient indexing using HNSW
 - System produces context-aware responses
-

Weaknesses

- Moderate answer relevancy (~0.5)
 - Inconsistent faithfulness
 - Recall varies for complex queries
 - Chunking may fragment structured content
 - LLM may over-explain or misinterpret
-